

Runtime Dark

ライブコーディングと実装解説に寄せた、黒基調の Hono Conference 用 Marp テーマです

2026 / Hono Conference Theme Prototype

01

ランタイムを見ながら話す

構成より先に、リクエストがどう流れるかを見せていきます

通常スライド

Hono の実装解説では、話す順番と画面の順番をなるべく近づけます

- 1スライド1メッセージで、視線を迷わせません
- 重要語は `Context` や `Middleware` のようにコード表記で出します
- 補足は短く、口頭で足せる余白を残します

コードを読む前に、どのリクエストを処理しているかだけ共有しておく と 追いやすい です

コードを主役にする

Hono の最小構成から、処理の差分をその場で積み上げます

```
import { Hono } from "hono";

const app = new Hono();

app.get("/health", (c) => {
  return c.json({
    ok: true,
    runtime: "edge",
  });
});

export default app;
```

2 カラムで「概念」と「実装」を並べる

見せたい考え方

- ルーティングは入口を細く保ちます
- `Context` にリクエストとレスポンスを集めます
- ミドルウェアで横断処理を外へ出します

```
app.use("*", logger());
app.use("/api/*", bearerAuth({ token }));

app.get("/api/posts/:id", async (c) => {
  const id = c.req.param("id");
  const post = await repo.find(id);

  return c.json(post);
});
```

ライブ実装のチェックポイント

01 request まず URL とメソッドを決めます

02 context 必要な値を `c.req` から取り出します

03 response 最後に `c.json()` で返します

```
$ curl localhost:8787/api/posts/42
```

```
200 {"id":"42","title":"Honoを使え"}
```

実装が見えると、理解が速くなります

小さく動かして、すぐ読めるコードにして、必要なところだけ深掘りしましょう!

Thank you / Runtime Dark